

PostgreSQL Performance Tuning Cheat Sheet

Top 20 postgresql.conf parameters with OLTP/OLAP recommendations, formulas, and gotchas.

Baseline System Info Needed

```
# How much RAM?
free -h
cat /proc/meminfo | grep MemTotal

# How many CPU cores?
nproc

# Storage type?
cat /sys/block/sda/queue/rotational # 0=SSD, 1=HDD
lsblk -d -o name,rota

# PostgreSQL version
psql -c "SELECT version();"
```

Top 20 Parameters — Reference Table

#	Parameter	OLTP	OLAP	Notes
1	shared_buffers	25% RAM	25% RAM	First cache layer
2	effective_cache_size	75% RAM	75% RAM	Planner hint
3	work_mem	16–64 MB	256 MB–1 GB	Per-sort, per-hash
4	maintenance_work_mem	256 MB	1–2 GB	VACUUM, CREATE INDEX
5	random_page_cost	1.1 (SSD)	1.1 (SSD)	Critical for index use
6	effective_io_concurrency	200 (SSD)	200 (SSD)	Parallel prefetch
7	wal_buffers	64 MB	64 MB	WAL write buffer
8	checkpoint_completion_target	0.9	0.9	Spread checkpoint I/O
9	checkpoint_timeout	5–10 min	15 min	Max checkpoint interval
10	max_wal_size	4 GB	8 GB	WAL disk limit
11	synchronous_commit	on	off	Durability vs. speed
12	max_connections	100–200	50–100	Use PgBouncer!
13	default_statistics_target	100	200–500	Planner accuracy
14	max_parallel_workers_per_gather	2	CPUs/2	Parallelism

15	max_parallel_workers	4	CPUs	Worker pool
16	autovacuum_vacuum_scale_factor	0.05	0.01	Vacuum frequency
17	autovacuum_max_workers	3	5	Concurrent vacuums
18	log_min_duration_statement	1000 ms	5000 ms	Slow query log
19	idle_in_transaction_session_timeout	60 s	300 s	Kill idle-in-txn
20	wal_level	replica	replica	Replication support

Detailed Parameter Guide

1. shared_buffers

```
shared_buffers = 4GB      # for 16GB RAM server
shared_buffers = 8GB      # for 32GB RAM server
shared_buffers = 16GB     # for 64GB RAM server
```

Formula: $\text{shared_buffers} = \text{RAM} \times 0.25$ **Maximum practical:** ~8–16 GB (OS page cache handles the rest)

Requires restart: Yes

GOTCHA: Do NOT set > 40% RAM. PostgreSQL also relies on the OS page cache as a second layer. Setting shared_buffers too high starves the OS cache.

2. effective_cache_size

```
effective_cache_size = 12GB    # for 16GB RAM server (75% of RAM)
```

Formula: $\text{effective_cache_size} = \text{RAM} \times 0.75$ **Does not allocate memory** — it's a **hint** to the planner about how much OS cache is available. **Requires restart:** No (pg_reload_conf())

3. work_mem

```
# OLTP
work_mem = 32MB

# OLAP / reporting
work_mem = 256MB

# Set per-session for heavy queries
SET work_mem = '1GB';
```

Formula for safe OLTP setting:

```
work_mem = (RAM × 0.25) / (max_connections × 2)
# Example: 16GB RAM, 100 connections:
```

```
# (16384 MB × 0.25) / (100 × 2) = 4096 / 200 = ~20 MB
```

Per-query risk: Complex queries can allocate `work_mem` × number of sorts/hashes running simultaneously

GOTCHA: `work_mem` is per operation (each sort, each hash join), per query plan node. A single complex query can use `work_mem` × 10 in the worst case. Multiply by `max_connections` for total exposure.

```
-- Detect sort spills (work_mem exceeded)
SELECT * FROM pg_stat_activity WHERE state = 'active' AND query LIKE '%Sort%';
EXPLAIN (ANALYZE, BUFFERS) SELECT ... ORDER BY complex_expression;
-- Look for: "Sort Method: external merge  Disk: NNNN kB"
```

4. maintenance_work_mem

```
maintenance_work_mem = 512MB    # OLTP
maintenance_work_mem = 2GB      # OLAP (large index builds)
```

Used by: VACUUM, CREATE INDEX, CLUSTER, ALTER TABLE, REINDEX **Formula:** Min(RAM × 0.1, 2GB) for production; higher for maintenance windows

5. random_page_cost

```
# SSD (most cloud instances)
random_page_cost = 1.1

# HDD
random_page_cost = 4.0  (default)

# Network-attached storage (S3-like)
random_page_cost = 4.0  (or higher)
```

THIS IS CRITICAL. Wrong value causes PostgreSQL to ignore indexes on SSD.

GOTCHA: Cloud DBs (RDS, Cloud SQL, Aurora) use SSDs but default is often 4.0. Explicitly set to 1.1 and watch query plans improve significantly.

6. effective_io_concurrency

```
effective_io_concurrency = 200    # SSD
effective_io_concurrency = 1      # HDD (rotational)
effective_io_concurrency = 300    # NVMe / RAID SSD
```

Tells PostgreSQL how many concurrent I/O operations the storage can handle. Used by bitmap heap scan and parallel queries.

7. wal_buffers

```
wal_buffers = 64MB      # most workloads (default -1 = 1/32 of shared_buffers, max 64MB)
```

Rarely needs tuning beyond 64MB. Auto-set to shared_buffers/32 up to 64MB.

8-10. Checkpoint Settings

```
checkpoint_completion_target = 0.9  # spread writes over 90% of checkpoint interval
checkpoint_timeout = 15min           # max time between checkpoints (OLAP)
max_wal_size = 4GB                  # WAL size before forced checkpoint
min_wal_size = 80MB                 # WAL minimum
```

Checkpoint tuning goal: Avoid "checkpoint burst" (all dirty pages written at once)

- `checkpoint_completion_target = 0.9` spreads writes over most of the interval
- `max_wal_size` controls how much WAL accumulates before a checkpoint is forced

GOTCHA: If you see `LOG: checkpoint was too frequent, happening every X seconds`, increase `max_wal_size` or `checkpoint_timeout`.

11. synchronous_commit

```
# Full durability (default)
synchronous_commit = on

# Async (lose max ~200ms of transactions on crash, but 10x faster writes)
synchronous_commit = off

# Per-transaction override (safe: most txns async, important ones sync)
SET LOCAL synchronous_commit = on; -- inside transaction
```

GOTCHA: `synchronous_commit = off` does NOT cause data corruption on crash — you only lose the last ~200ms of commits. The database remains consistent. Good for: logging, analytics, session data.

12. max_connections

```
max_connections = 100      # per PostgreSQL recommendation for most workloads
```

ALWAYS use PgBouncer (connection pooler). Raw PostgreSQL connections:

- Each connection = ~5-10 MB RAM
- PostgreSQL was not designed for thousands of persistent connections
- `max_connections = 100` with PgBouncer pooling 1000 app connections = ideal

PgBouncer modes:

transaction pooling (recommended): connection released after transaction

```
session pooling: connection released after client disconnect
statement pooling: fastest but breaks multi-statement transactions
```

13. default_statistics_target

```
default_statistics_target = 100    # default
default_statistics_target = 200    # OLAP, complex queries
default_statistics_target = 500    # when estimates are badly wrong
```

Per-column override (better than global change):

```
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 200;
ALTER TABLE events ALTER COLUMN user_id SET STATISTICS 500;
ANALYZE orders;
```

GOTCHA: Higher statistics_target = ANALYZE takes longer and pg_statistic uses more space. Only increase for columns with poor estimates.

14–15. Parallelism

```
max_parallel_workers_per_gather = 4    # OLAP (half of CPUs)
max_parallel_workers = 8               # total (= num CPUs)
max_parallel_maintenance_workers = 4   # for CREATE INDEX parallel
parallel_setup_cost = 1000             # lower if parallel underused
parallel_tuple_cost = 0.1
min_parallel_table_scan_size = 8MB
min_parallel_index_scan_size = 512kB
```

Formula for OLAP: `max_parallel_workers_per_gather = CPUs / 2`

```
-- Force parallel for a query
SET max_parallel_workers_per_gather = 8;
SET force_parallel_mode = on; -- testing only

-- Check if parallel is used
EXPLAIN SELECT ... FROM large_table;
-- Look for: "Gather" or "Gather Merge" node
```

16–17. Autovacuum Tuning

```
# For write-heavy OLTP tables
autovacuum_vacuum_scale_factor = 0.05    # vacuum when 5% of table is dead
autovacuum_analyze_scale_factor = 0.02   # analyze when 2% changed
autovacuum_vacuum_threshold = 100        # minimum rows before considering vacuum
autovacuum_max_workers = 5               # more workers for write-heavy
```

```
# For very large tables (per-table override):
```

```
-- Per-table autovacuum override (most effective approach)
ALTER TABLE large_events SET (
    autovacuum_vacuum_scale_factor = 0.01,    -- vacuum when 1% dead
    autovacuum_vacuum_threshold = 1000,
    autovacuum_analyze_scale_factor = 0.005,
    autovacuum_vacuum_cost_delay = 2
);

-- Emergency manual vacuum
VACUUM ANALYZE large_events;
VACUUM VERBOSE large_events; -- see what it does
```

GOTCHA: Autovacuum is critical. Never disable it. If autovacuum can't keep up, tables bloat, statistics go stale, and you risk XID wraparound (catastrophic). Monitor `pg_stat_user_tables.n_dead_tup` and `n_mod_since_analyze`.

18. Slow Query Logging

```
log_min_duration_statement = 1000    # log queries > 1 second
log_checkpoints = on
log_connections = off                # too noisy with PgBouncer
log_lock_waits = on                  # log slow lock waits
log_temp_files = 0                   # log all temp file creation (work_mem spills)
log_autovacuum_min_duration = 250    # log slow autovacuums
```

19. Timeout Settings

```
idle_in_transaction_session_timeout = 60000    # 60 seconds (ms)
statement_timeout = 0                          # 0 = no limit (set per-role)
lock_timeout = 0                               # 0 = no limit (set per-role)
```

```
-- Per-role timeout (safer than global)
ALTER ROLE app_user SET statement_timeout = '30s';
ALTER ROLE reporting_user SET statement_timeout = '300s';

-- Per-session
SET statement_timeout = '10s';
```

20. WAL Level

```
wal_level = replica    # supports streaming replication + logical replication
wal_level = logical     # also supports logical replication slots (Debezium, etc.)
```

```
wal_level = minimal      # no replication possible (only for standalone nodes)
```

Complete Config Templates

OLTP Production Config (16GB RAM, 8 CPU, SSD)

```
# Memory
shared_buffers = 4GB
effective_cache_size = 12GB
work_mem = 32MB
maintenance_work_mem = 512MB
wal_buffers = 64MB

# Storage
random_page_cost = 1.1
effective_io_concurrency = 200
seq_page_cost = 1.0

# WAL / Checkpoints
wal_level = replica
synchronous_commit = on
checkpoint_completion_target = 0.9
checkpoint_timeout = 10min
max_wal_size = 4GB
min_wal_size = 80MB

# Connections
max_connections = 100
superuser_reserved_connections = 3
idle_in_transaction_session_timeout = 60000

# Planner
default_statistics_target = 100
max_parallel_workers_per_gather = 2
max_parallel_workers = 4

# Autovacuum
autovacuum = on
autovacuum_max_workers = 3
autovacuum_vacuum_scale_factor = 0.05
autovacuum_analyze_scale_factor = 0.02
autovacuum_vacuum_cost_delay = 2ms

# Logging
log_min_duration_statement = 1000
log_checkpoints = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 250ms
```

OLAP / Data Warehouse Config (64GB RAM, 32 CPU, SSD)

```
# Memory
shared_buffers = 16GB
effective_cache_size = 48GB
work_mem = 256MB          # higher for large aggregations
maintenance_work_mem = 2GB
wal_buffers = 64MB

# Storage
random_page_cost = 1.1
effective_io_concurrency = 200

# WAL
wal_level = replica
synchronous_commit = off  # analytics writes can be async
checkpoint_completion_target = 0.9
checkpoint_timeout = 30min
max_wal_size = 8GB

# Connections
max_connections = 50      # fewer; use connection pools

# Planner
default_statistics_target = 300
max_parallel_workers_per_gather = 8
max_parallel_workers = 16
max_parallel_maintenance_workers = 8

# Autovacuum (less aggressive for read-heavy OLAP)
autovacuum_vacuum_scale_factor = 0.1
autovacuum_analyze_scale_factor = 0.05

# Logging
log_min_duration_statement = 5000
```

Performance Tuning Checklist

Step 1: Find Slow Queries

```
CREATE EXTENSION pg_stat_statements;
SELECT substring(query,1,80), calls, round(total_exec_time/calls::numeric,2) avg_ms,
       round(total_exec_time::numeric,0) total_ms
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;
```

Step 2: Analyze Query Plans


```
EXPLAIN (ANALYZE, BUFFERS) <slow query>;
-- Look for: Seq Scan on large tables, bad row estimates, sort spills
```

Step 3: Check Index Usage

```
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes WHERE idx_scan = 0;
SELECT tablename, seq_scan, idx_scan FROM pg_stat_user_tables ORDER BY seq_scan
DESC;
```

Step 4: Check Table Bloat

```
SELECT relname, n_dead_tup, n_live_tup,
       round(100.0*n_dead_tup/nullif(n_live_tup+n_dead_tup,0),2) AS dead_pct
FROM pg_stat_user_tables WHERE n_dead_tup > 10000 ORDER BY dead_pct DESC;
```

Step 5: Check Autovacuum

```
SELECT relname, last_autovacuum, last_autoanalyze, n_dead_tup, n_mod_since_analyze
FROM pg_stat_user_tables WHERE n_dead_tup > 10000;
```

Step 6: Check Cache Hit Ratio

```
SELECT
    heap_blks_read, heap_blks_hit,
    round(100.0*heap_blks_hit/nullif(heap_blks_hit+heap_blks_read,0),2) AS
cache_hit_pct
FROM pg_statio_user_tables
WHERE heap_blks_hit + heap_blks_read > 0
ORDER BY cache_hit_pct ASC;
-- Target: > 99% for OLTP
```

Step 7: Check Connection Count

```
SELECT count(*), state FROM pg_stat_activity GROUP BY state;
SELECT max_conn, used, res_for_super, max_conn - used - res_for_super AS available
FROM
    (SELECT count(*) used FROM pg_stat_activity) t,
    (SELECT setting::int max_conn FROM pg_settings WHERE name='max_connections') t2,
    (SELECT setting::int res_for_super FROM pg_settings WHERE
name='superuser_reserved_connections') t3;
```